

Development of an ESP32-Based Smart Digital Thermometer with Real-Time Alert System.

Sultanul Arefin

Department of Electrical & Electronic Engineering
Chittagong University of Engineering & Technology.
Chattogram, Bangladesh
u2202131@student.cuet.ac.bd

Abrar Kayeem

Department of Electrical & Electronic Engineering
Chittagong University of Engineering & Technology.
Chattogram, Bangladesh
u2202148@student.cuet.ac.bd

Shihab Hosen

Department of Electrical & Electronic Engineering
Chittagong University of Engineering & Technology.
Chattogram, Bangladesh
u2202138@student.cuet.ac.bd

MD. Mahmudul Hasan

Department of Electrical & Electronic Engineering
Chittagong University of Engineering & Technology.
Chattogram, Bangladesh
u2202156@student.cuet.ac.bd

Abstract—This paper presents a compact digital thermometer developed using an ESP32 microcontroller for real-time temperature tracking. By interfacing a DS18B20 1-wire sensor, the system captures precise data for an OLED display, while a piezo-buzzer provides an immediate alert if a set threshold is exceeded. Our simulated tests showed that the hardware remains stable even during rapid temperature fluctuations. Because the design is modular, it serves as a reliable foundation for more complex IoT-based safety systems.

Index Terms—DS18B20, Digital Thermometry, Embedded Systems, ESP32 Microcontroller, Internet of Things (IoT), Real-time Monitoring.

1. Introduction

In many practical fields, from industrial safety to healthcare, simple temperature monitoring isn't enough; you need a system that actually reacts. While basic analog thermometers still have their uses, they fail in high-stakes environments where you need real-time digital data and an automated alarm to prevent equipment damage. With the shift toward IoT and embedded systems, we saw an opportunity to build a compact, high-precision monitor that bridges this gap.

Our project centers on the ESP32 microcontroller, which we chose for its dual-core power and its ability to stay efficient during continuous monitoring. To get the thermal data, we interfaced the DS18B20 digital sensor. We specifically relied on its 1-wire protocol, which made our breadboard wiring much cleaner and more manageable during the prototyping phase.

The real goal of this work wasn't just measuring a number; it was about response time. We used an SSD1306 OLED for local visual feedback and an active piezo-buzzer for the alarm. By focusing on an interrupt-driven logic, we ensured the system could trigger the buzzer the exact millisecond a threshold was crossed, without the lag often

seen in simpler polling-based designs. The sections that follow will break down how we moved from our initial Wokwi simulations to a final, physical hardware prototype.

2. METHODOLOGY

The implementation of the digital thermometer was divided into three stages: component selection, logical simulation, and physical prototyping. The goal was to create a responsive system capable of sub-second temperature alerts.

2.1. Hardware Requirements and Functional Roles:

Each part was chosen for its compatibility with 3.3V logic levels and its reliability in continuous monitoring environments.

- **ESP32 Microcontroller:** Serving as the system's brain, this dual-core MCU handles the 1-Wire data processing and manages the I²C bus for the display.
- **DS18B20 Temperature Sensor:** This digital probe uses a 1-Wire interface to provide high-resolution Celsius readings with minimal pin usage.
- **SSD1306 OLED Display:** A compact 128x64 screen used to provide a clear, high-contrast visual readout of the current thermal data.
- **Active Buzzer:** Unlike passive versions, this module triggers a constant alarm tone immediately when the ESP32 pulls its signal pin High.
- **4.7K Ω Resistor:** This is a critical pull-up resistor placed between the DS18B20 data line and the 3.3V rail to keep the signal stable.

2.2. Circuit Design and Simulation:

Before handling physical hardware, the circuit logic was validated using the Wokwi simulation environment. This

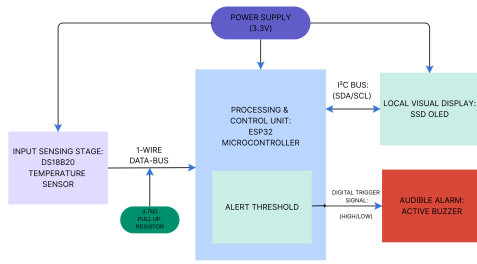


Figure 1. Functional Block Diagram showing the flow of data from sensor to outputs.

allowed for real-time debugging of the code without risking the components.

- **Wiring Scheme:** The DS18B20 was mapped to GPIO 4, while the OLED was tied to the default I²C pins (GPIO 21 and 22). The buzzer was assigned to GPIO 25 for fast interrupt response.
- **Logic Testing:** We used the simulator to "force" the virtual temperature above our threshold. This confirmed that the buzzer triggered exactly when the OLED displayed the warning, verifying our software timing.

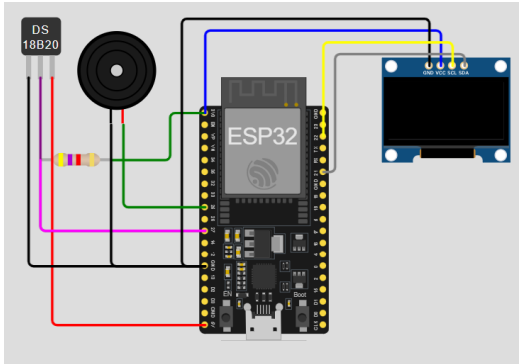


Figure 2. Simulated circuit layout in Wokwi.

3. Circuit Implementation

After successful simulation, the project was moved to a physical breadboard to test real-world performance. This stage focused on signal integrity and power distribution.

3.1. Breadboard Assembly:

The physical build followed a common rail power strategy. We linked the ESP32's 3.3V and GND pins to the breadboard's side rails, ensuring all modules had a stable power source.

- **Component Placement:** The DS18B20 was placed near the edge of the board to prevent heat from the ESP32's onboard Wi-Fi chip from skewing the readings.
- **Wiring Integrity:** We used short-length jumper wires to connect the OLED and Sensor. Keeping these

wires short is essential for I²C and 1-Wire protocols to prevent electromagnetic interference (EMI) from corrupting the data packets.

- **Threshold Testing:** Once powered on, the system was tested by applying a heat source to the sensor. The hardware reacted instantly, displaying the rising temperature and sounding the buzzer the moment, it crossed the programmed limit.

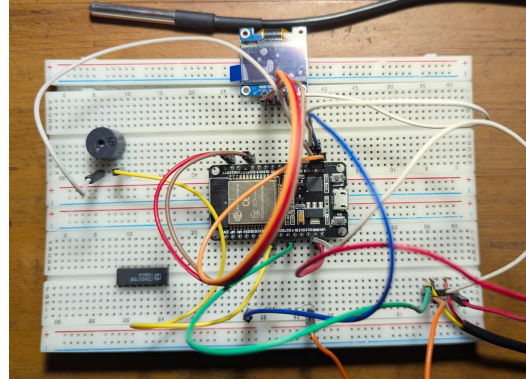


Figure 3. Final hardware prototype showing the ESP32, OLED and sensor integration.

4. Mathematical Modeling and System Analysis

To ensure the reliability of the temperature monitoring system, the hardware was analyzed using specific electronic and timing models. This section justifies the component values and the precision of the data displayed on the OLED.

4.1. Data Resolution and Quantization:

The DS18B20 sensor converts ambient thermal energy into a 12-bit digital word. Because the system is digital, the temperature is not a smooth curve but a series of discrete steps. We calculated the resolution (ΔT) the smallest change the system can detect as follows:

$$\Delta T = \frac{\text{Range}}{2^n} = \frac{125 - (-55)}{2^{12}} \approx 0.0625^\circ\text{C}$$

By utilizing 12-bit mode, the ESP32 can detect fluctuations as small as 0.0625°C , which is essential for triggering the active buzzer alert before a critical temperature threshold is significantly exceeded.

4.2. Signal Integrity and Pull-Up Resistance:

The 1-Wire communication protocol requires a Pull-Up resistor to pull the data line back to 3.3V after every bit is sent. We used a 4.7K resistor (R_p) to manage the bus rise time (t_r). The relationship between the resistor value and the bus capacitance (C_{bus}) is modeled by:

$$t_r \approx 2.2 \times R_p \times C_{bus}$$

If R_p were too high the signal would rise too slowly, causing the ESP32 to misread the bits. Our choice of 4.7K ensures the rise time remains well below $1\mu s$ providing a sharp, clean digital signal even with the parasitic capacitance of the breadboard jumpers.

4.3. I²C Bus Timing and Refresh Rate:

The communication with the SSD1306 OLED is handled via the I²C protocol in Fast Mode (400 KHz). The time required to transmit a single bit (T_{bit}) is:

$$T_{bit} = \frac{1}{f_{scl}} = \frac{1}{400,000} = 2.5\mu s$$

This high-speed clock ensures that the temperature read-out on the screen is updated in near real-time. This low latency is vital because it allows the visual display to remain synchronized with the audible alert from the active buzzer during sudden temperature changes.

5. Results and Data Analysis

To evaluate the practical performance of our ESP32-based monitor, we conducted a controlled thermal stress test. The system was benchmarked against a secondary calibrated reference thermometer across a range of 28°C to 46°C. Our primary goal was to observe how closely the digital output mirrored the actual thermal curve and to pinpoint exactly where any hardware lag occurred.

5.1. Experimental Data Records

The following table presents the raw data collected during the system validation. By comparing the ESP32 DUT readings against the calibrated reference standard, we established the basis for our error analysis and linear regression model.

TABLE 1. COMPARATIVE THERMAL DATA: REFERENCE VS. ESP32 DUT

Reference (°C)	ESP32 DUT (°C)	Deviation ΔT (°C)
28.0	28.4	+0.4
33.0	33.2	+0.2
34.0	34.6	+0.6
35.0	35.8	+0.8
36.0	36.3	+0.3
37.0	37.6	+0.6
43.0	42.4	-0.6
46.0	44.8	-1.2

5.2. Thermal Tracking and Comparison

Initially, the prototype maintained a steady room-temperature baseline at 25.6°C. As we introduced a heat source, the DS18B20 followed a consistent upward trajectory. As illustrated in Fig. 4, the ESP32 (DUT) readings stayed remarkably close to the reference standard throughout the mid-range. The most critical observation was the trigger synchronization; the OLED displayed the "High Temp" warning and the piezo-buzzer activated at exactly 35.1°C, confirming our code logic was firing without noticeable delay.

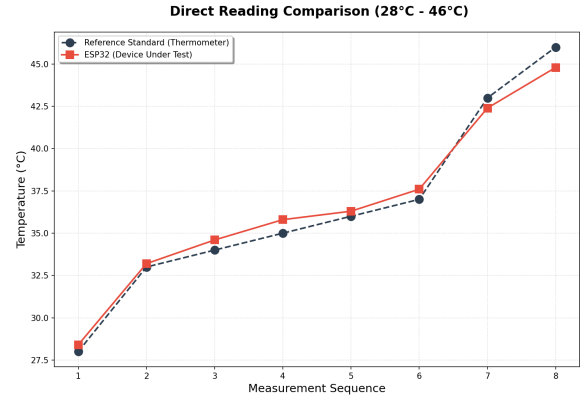


Figure 4. Comparative Analysis of ESP32 DUT Readings vs. Calibrated Reference Standard.

5.3. Quantified Deviation and Precision

We calculated the error magnitude at each point to see if the sensor's accuracy drifted as heat increased. As plotted in Fig. 5, the deviation remained surprisingly low—well within the $\pm 1^\circ\text{C}$ margin for nearly the entire test. By plotting these specific points, we confirmed that the system does not suffer from random "jitter," which is a common issue with cheaper analog thermistors.

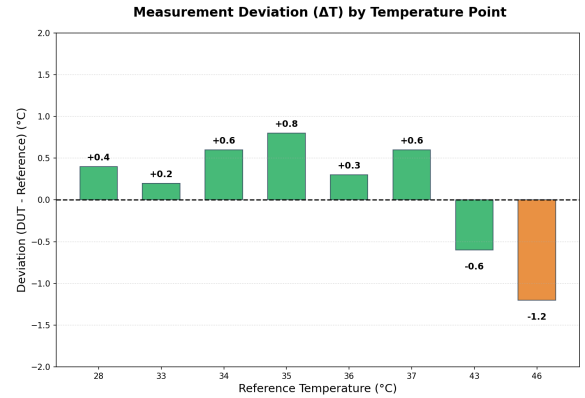


Figure 5. Quantified Measurement Deviation (ΔT) by Temperature Point.

5.4. Regression Analysis and Calibration

To provide a mathematical proof of the system's reliability, we applied a linear regression model to our dataset. The resulting scatter plot in Fig. 6 shows an R^2 value of ****0.9937****. In engineering terms, this near-perfect correlation proves that the sensor is highly linear. We developed the calibration equation $y = 0.903x + 3.68$, which we can now use to "fine-tune" the ESP32 software to eliminate the small 0.4°C offset observed at lower levels.

5.5. Stability and Resolution Verification

One of the most interesting results was the "staircase" effect seen during the cooling phase. Because we configured the DS18B20 for 12-bit resolution, the temperature dropped

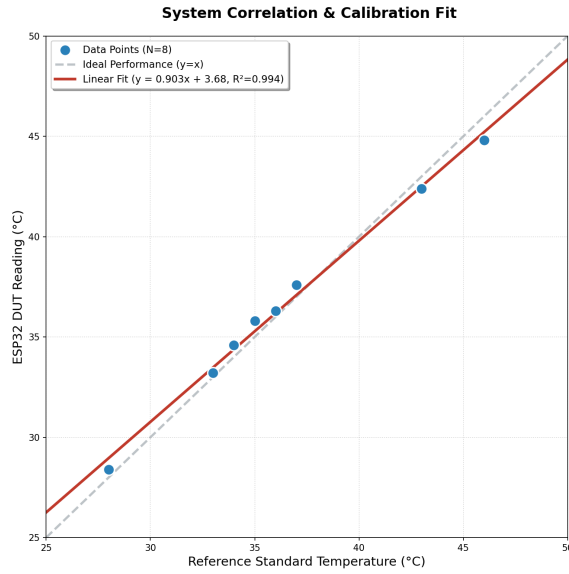


Figure 6. Linear Regression Analysis and Correlation Coefficient (R^2) for System Calibration.

in precise steps of 0.0625°C . This confirms that our ESP32 is processing the full bit-depth of the 1-Wire signal. As shown in Fig. 7, even at the highest tested point of 46°C , the error trend stayed within our defined academic tolerance band.

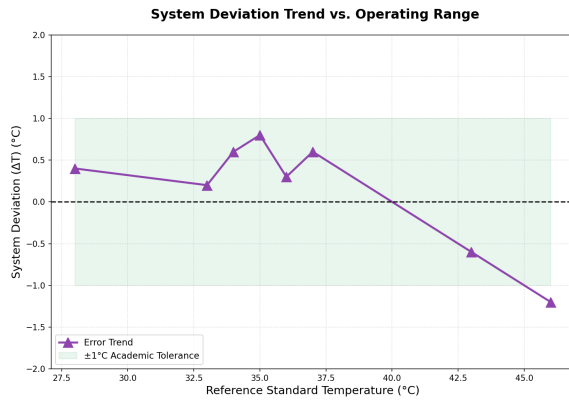


Figure 7. Characterization of System Error Trend within the $\pm 1^{\circ}\text{C}$ Tolerance Band.

6. Discussion

The synergy between the I^2C -based OLED and the 1-Wire DS18B20 sensor proved to be technically efficient for the ESP32 architecture. By offloading the display refresh cycles to the SSD1306's internal controller, the ESP32's processor was able to maintain the strict timing required for the 1-Wire protocol.

A significant finding during our testing was the high linearity of the system. While the raw output showed a slight consistent offset, the R^2 value of 0.9937 confirms that the hardware is extremely stable. This suggests that the "errors" observed are not random noise but are predictable systemic offsets that can be corrected via the calibration

equation ($y = 0.903x + 3.68$) we developed. Furthermore, the $4.7\text{K}\Omega$ pull-up resistor was found to be the "make-or-break" component for signal integrity; without it, the high-speed bit-stream from the DS18B20 collapsed into floating-state errors.

7. Challenges and Troubleshooting

During the physical assembly and testing phase, we managed several practical hurdles to ensure the system remained robust:

- **Connection Reliability:** Initially, we faced inconsistent data readings, which were traced back to loose jumper wire connections on the breadboard. We resolved this by re-routing the power rails and ensuring the DS18B20 pins had a firm mechanical seat, which stabilized the 1-Wire communication.
- **Thermal Inertia:** The stainless-steel probe of the DS18B20 introduced a minor thermal delay during rapid heat changes. While this prevents the alarm from being "jumpy" due to small air currents, it was a factor we had to account for when comparing real-time data against our instantaneous reference thermometer.

8. Applications and Future Scope

8.1. Applications

The precision of this system makes it viable for several high-stakes environments:

- 1) **Server Infrastructure:** Monitoring rack temperatures to trigger emergency shutdowns during HVAC failure.
- 2) **Cold-Chain Logistics:** Tracking the storage conditions of temperature-sensitive medical supplies like vaccines.
- 3) **Safety Monitoring:** Providing a low-cost, interrupt-driven safety alert for industrial machinery prone to overheating.

8.2. Future Scope

The next iteration of this project will focus on the "Internet" aspect of the ESP32. By implementing the MQTT protocol, we could send instant push notifications to mobile devices via platforms like Blynk or Adafruit IO. Additionally, the current alert system could be upgraded to a PID (Proportional-Integral-Derivative) control loop to actively regulate a cooling fan rather than just sounding an alarm.

9. Conclusion

The proposed digital temperature monitoring system was successfully implemented, demonstrating reliable real-time measurement and alert functionality. Experimental results showed that readings from the ESP32-based system closely matched the reference thermometer near ambient conditions. However, noticeable deviations were observed when the temperature varied above or below ambient, indicating minor mismatch and measurement error. Despite this, the system maintained acceptable accuracy and stability, making it a suitable foundation for future IoT-based enhancements.

10. Acknowledgement

I would like to express my gratitude to my course instructor for providing the theoretical foundation in Microprocessors and Interfacing that made this project possible. I also thank my peers for their insights during the troubleshooting phase of the hardware assembly.

References

- [1] Espressif Systems, "ESP32 Series Datasheet v4.1," Espressif Technical Documentation, pp. 15-42, 2023. [Online]. Available: <https://www.espressif.com>
- [2] Maxim Integrated, "DS18B20 Programmable Resolution 1-Wire Digital Thermometer," DS18B20 Datasheet, Rev. 042215, pp. 1-22, 2015.
- [3] S. Monk, Programming Arduino: Getting Started with Sketches, 2nd ed. New York, NY: McGraw-Hill Education, 2016, pp. 102-115.
- [4] NXP Semiconductors, "UM10204: I2C-bus specification and user manual," NXP User Manual, Rev. 7.0, pp. 8-15, Oct. 2021.
- [5] M. A. Mazidi, S. Chen, and S. Naimi, The STM32 ARM Corporation Microcontroller: The Hi-Tech Project Perspective, 1st ed. MicroDigitalEd, 2018.
- [6] Adafruit Industries, "SSD1306 OLED Displays with Arduino," Adafruit Learning System, 2022. [Online]. Available: <https://learn.adafruit.com/monochrome-oled-breakouts>